MDP 정식화 (Markov Decision Processes)

MDP의 다섯 요소와 마르코프 성질 · 누적 보상과 할인율 · 가치함수와 벨만 기대방정식

Machine Learning 2 · 3주차

한국과학영재학교 (KSA)

Week 3

이번 주차 개요

1950년대 벨만(Richard Bellman, 1957)은 “차원의 저주”라는 용어를 만들며 **동적계획법**을 고안. 핵심 통찰 — **복잡한 문제를 한 번에 풀지 말고, 작은 부분 문제로 쪼개 그 답을 재귀적으로 조합하라** — 가 지금 강화학습 이론 전체의 뼈대.

- Week 2의 밴딧은 **상태가 정확히 하나뿐인, 전이도 없는 특수한 MDP**.
- 이번 장은 밴딧에 **상태**와 **전이**를 더해, “지금 행동이 다음 상태를 바꾸고, 그 상태가 미래 보상을 결정한다” 는 순환을 수학적으로 닫는다.

이 주차를 마치면

- MDP의 다섯 요소 $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ 를 한 줄로 쓰고, **마르코프 성질**이 무엇을 요구하는지 판단한다.
- 할인된 누적 보상(리턴) G_t 를 손으로·코드로 계산하고, γ 의 두 얼굴(수렴 장치 + 설계 변수)을 설명한다.
- 벨만 기대방정식이 무한합을 “한 스텝짜리 재귀식”으로 압축하는 과정을 따라간다.
- 반복 대입이 바로 다음 장 **정책평가** 알고리즘임을 확인한다.

세 차시

1. **3.1 MDP의 다섯 요소와 마르코프 성질** – “게임의 규칙 전체”를 다섯 요소로 닫고, 3칸 보드게임 MDP를 통째로 써본다.
2. **3.2 누적 보상, 할인율, 그리고 실습** – 리턴 G_t 계산, **유효 시계**, “γ 가 목표를 바꾼다”를 확인한다.
3. **3.3 가치함수와 벨만 기대방정식** – V^π, Q^π 정의, 무한합을 한 스텝 재귀식으로 압축하고, 반복 대입을 예고한다.

이 “문법”을 확실히 잡아야, Week 4 정책평가 · Week 6 Q-러닝 · Week 9 DQN 손실함수가 사실 **동일한 재귀식(벨만방정식)**을 각각 다른 방식으로 계산·학습하는 것임을 뒤에서 알아볼 수 있다.

3.1 MDP의 다섯 요소와 마르코프 성질

이 절에서 무엇을 만들 것인가

1. MDP는 다섯 가지 요소 $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ 하나로 게임의 규칙을
2. 마르코프 성질의 정확한 의미와 깨지는 경우

MDP의 다섯 요소

MDP는 다섯 가지로 정의된다: $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$

요소를 한 줄로	의미
$\mathcal{S}$	가능한 **상태**들의 집합
$\mathcal{A}$	가능한 **행동**들의 집합
$P(s'\|s,a)$	s에서 a를 했을 때 s'로 **전이**될 확률
$R(s,a)$	s에서 a를 했을 때 받는 **즉시 보상**
$\gamma \in [0,1)$	**할인율** (discount factor)

Week 2의 밴딧과 비교

밴딧에는 $\mathcal{S}$도 P도 **없었다**(팔을 당겨도 "다음 상태"가 없었다). MDP는 지금 한 행동이 다음 상태 $P(s'|s,a)$를 통해 **미래 전체**에 영향을 준다는 사실을 명시적으로 담는다.

“다섯 개면 충분한가?” — 상태가 화폐

- 보상은 (s, a) 에서, 다음 상태는 (s_t, a_t) 에서 결정된다.
- 즉 **상태만 알면 앞으로 일어날 것의 전체 확률법칙이 결정된다**.

이 문장이 바로 **“마르코프 성질”이라고 부르는 것의 정확하고 강한 형태**(다음 섹션에서 정식화).

다섯 요소가 “게임의 규칙 전체”를 담는다는 뜻

초기 상태 분포는 **알고리즘에 필요한 정보가 아니다** — 학습은 어떤 시작점에서든 똑같이 진행되며, 그 점은 연습문제 4에서 확인한다.

역사: 이름이 붙은 두 수학자, 반세기의 드라마

마르코프(Andrei Markov, 1856–1922)

- 확률론과 수론을 오가던 러시아 수학자.
- 1906년 논문: 당시 확률의 대명사 **베르누이 법칙** (독립 반복 시행의 상대빈도 $\rightarrow$ 확률)의 왕좌를 지키던 확률론에 **의존하는 시행**까지 포괄.
- "이번 결과가 직전 결과만 기억하고 이전은 잊는" 의존성(= 지금의 마르코프 체인)에서도 베르누이 법칙이 성립함을 보임.

의존성이 들어와도 법칙이 살려면 그 의존성은 **1단계만**이어야 한다는 발견이, 120년 뒤 강화학습에서 "상태 하나만 기억하면 된다"는 설계 원칙으로 직접 이어진다.

벨만(Richard Bellman, 1920–1984)

- 2차전지 중 미공군 폭격 편성 문제를 풀면서 발견: **최적 전략은 부분 문제의 최적 전략을 재귀적으로 조합할 수 있다**(최적성의 원리).
- 1957년 저서 Dynamic Programming에서 세운 프레임워크가 바로 MDP의 정식화.

흥미로운 우연: 마르코프 성질 논문(1906)과 MDP 정식화(1957) 사이는 50년, 그런데 두 사람의 문제는 **정확히 같은 문제의 양쪽 끝**이다.

같은 문제의 양쪽 끝

마르코프: "이 의존 구조가 어떤 **확률법칙**을 결정하는가"(기술, description)
벨만: "그 법칙 위에서 **최적 행동**을 어떻게 찾는가"(최적화, optimization)
이번 학기의 구조(이번 장 = 정식화 $\rightarrow$ Week 4–6 = 최적화)가 이 드라마의 둘째막.



Andrei Markov (1886)



Richard Bellman (1920–1984)

한 편의 MDP를 통째로 쓰기: 3칸 보드게임

3칸 보드게임 MDP 설정

- $\mathcal{S} = \{0, 1, 2\}$ — 보드의 세 칸. 2는 **터미널 상태**(terminal state, 게임 종료 상태).
- $\mathcal{A} = \{\text{right}, \text{wait}\}$ — 두 행동.
- $\gamma = 0.9$.

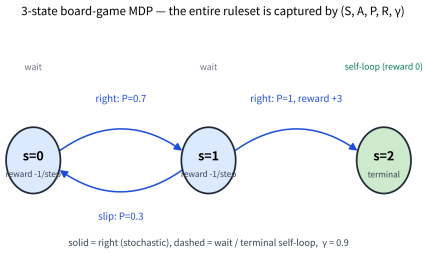
3칸 보드게임: 전이확률 P 와 보상 R

전이확률 P

상태	행동	s' (확률)
0	right	1 (0.7), 0 미끄러짐 (0.3)
0	wait	0 (1)
1	right	2 (1)
1	wait	1 (1)
2 (터미널)	(모든)	2 (1) self-loop

보상 R

- 터미널로 들어갈 전이(1에서 right): **+3**
- 그 외 모든 스텝: -1 (한 스텝 움직이는 데 드는 “시간 벌금”)



3상태 보드게임 MDP — 상태 0에서 시작, right(실선)/wait(점선) 전이, 미끄러짐 확률 0.3, 터미널 2에 진입 시 +3(그 외 매 스텝 -1).

각 요소 하나씩 (1/2): $\mathcal{S}$, $\mathcal{A}$

$\mathcal{S}$, 상태 공간

- 상태는 “에이전트의 관점에서의 **세계의 완전한 기술**”이어야 한다.

$\mathcal{A}$, 행동 공간

- 이산: CartPole의 left/right, FrozenLake의 4방향.
- 연속: Pendulum의 $[-2, 2] \subset \mathbb{R}$ (힘의 크기 그 자체를 고른다).

각 요소 하나씩 (2/2): P, R, γ

P, 전이확률

- **모델 기반 vs 모델 없는**

R, 보상

할인율: 유계 함수 존재 조건

마르코프 성질 (Markov property)

정의

마르코프 성질: 다음 상태는 오직 **현재** 상태와 행동에만 의존한다 — 어떻게 지금 상태에 도달했는지(과거 전체 이력)는 상관없다:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0) \;=\; P(s_{t+1} \mid s_t, a_t)$$

$$P(s_{t+1} = s' \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots) \;=\; P(s_{t+1} = s' \mid s_t, a_t)$$

마르코프 성질 정의

왜 이 성질이 알고리즘의 모든 것을 결정하는가

마르코프 성질이 "좋은 가정"을 넘어서 **필요 불가결**인 이유. **성질이 없다면**:

상태가치가 정의되지 않는다

- $V^{\pi}(s)$ 는 "상태 s 에서 시작"의 리턴 기댓값.
- "같은 s"에 과거 이력이 **다른** 두 경로가 존재하고 그 미래 분포가 다르면, s 만으로 인자화한 하나의 함수 $V(s)$ 로 그 둘을 동시에 설명할 수 없다.
- 같은 자리에 "두 개의 다른 가치"가 공존하는데, 표 기반 알고리즘은 **한 칸에 한 값**만 쓸 수 있다.

학습이 수렴하지 않는다

- 같은 s 를 매번 **다른 "진짜 상황"**의 대리인으로 쓰면, 추정치는 서로 다른 답을 향해 계속 튀어 다닌다.
- Week 6–11의 모든 수렴 증명은 "같은 s 는 같은 미래 분포를 갖는다"는 **이 동일성** 위에 서 있다.

반대로: 성질이 성립하면

상태 하나가 **과거 전체를 압축하는 무손실 요약**이 된다 — 통계학의 표현을 빌리면, 상태는 미래에 대한 과거 이력의 **충분 통계량(sufficient statistic)** 역할을 한다. 이 성질 덕분에 "기억의 길이"가 문제에서 빠져나가고, **현재 관측** 하나로 의사결정이 닫힌다.

마르코프 성질이 성립하지 않을 때: hidden state와 재설계

현실은 대부분 **부분관측**(partially observed)이다 — 에이전트가 보는 "관측"이 진짜 상태의 완전한 투영이 아니다.

상태 재설계의 두 가지 표준 전략

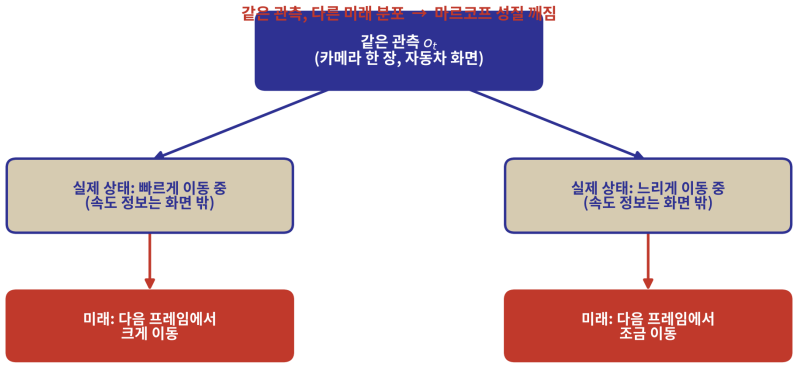
전략 1: 상태에 정보를 직접 넣는다 — 최근 k 프레임 $(o_t, o_{t-1}, \ldots, o_{t-k+1})$ 을 하나의 "확장 상태"로 묶어 s_t 로 재정의. k 가 충분하면 **속도 정보가 (프레임 차이로) 복원**되어 성질이 다시 성립. 해석은 쉽지만 "k 를 몇으로 할지"라는 **손으로 고르는 하이퍼파라미터**가 생긴다.

전략 2: 학습된 은닉 상태로 요약한다 — 신경망(예: RNN의 은닉 상태 h_t, Week 11)이 과거 관측 시퀀스를 h_t 로 압축하고 (o_t, h_t) 를 상태로 쓴다. "요약에 충분한 정보를 담았는지"는 수학적으로 검증할 수 없고 **경험적으로** 판단. 이것이 **부분관측 MDP(POMDP)** 라는, 이번 학기에서 직접 다루지는 않지만 **이름은 꼭 알아 둘** 문제 영역.

핵심

마르코프 성질은 "자연이 주는 성질"이 아니라, **우리가 상태를 어떻게 정의하는가의 설계 선택**이다. 성질이 "깨져" 보이면, 그건 우리의 상태 정의가 **부족했다는 신호**일 뿐, 강화학습 자체를 포기해야 할 이유가 아니다.

그림으로: 부분관측이 마르코프 성질을 깨는 방식



같은 카메라 관측 o_t 아래 “빠르게 이동 중”과 “느리게 이동 중”이라는 서로 다른 실제 상태가 숨어 있으면, 같은 o_t 에서 출발해도 다음 프레임의 분포가 달라진다 — 이것이 마르코프 성질이 깨지는 구체적 방식.



실제 자율주행 테스트 차량의 카메라·라이다 센서 클로즈업 (Zoox 테스트 차량, 2025년 5월, 샌프란시스코)
— 카메라 한 장만으로는 속도 같은 hidden state가 관측에 담기지 않는다.

확인 문제: MDP인가, 마르코프인가? (1/2)

네 시나리오 각각에 대해 (a) MDP로 모델링할 수 있는가, (b) 가능하다면 $\mathcal{S}, \mathcal{A}, P, R$ 을 한 줄씩 쓰라:

- **(a) 5층 건물의 엘리베이터 호출 제어**
  - 답: **MDP** — $\mathcal{S}$ = (각 층의 상/하 호출 버튼 상태 2^{10} 조합) $\times$ (엘리베이터 현재 층 5) = 5120개 이하, $\mathcal{A} = \{up, down, open\}$, P = 버튼 상태 + 층 이동, $R = -1$(매 스텝 벌금) 또는 "누락 호출" 페널티.
  - 마르코프 성질은 **성립**(현재 호출 상태가 미래의 모든 것을 결정).
- **(b) "오늘 기분"을 몰라서, 어제 몇 시간 폰을 봤는 것만 관측으로 가지는 추천 문제**
  - 답: **마르코프 성질이 깨진** 문제 — "기분"이 **hidden state**.
  - 어제 폰 사용 시간만으로는 같은 관측 아래 "피곤한 날"과 "재밌는 날"을 구별 못 해 **추천의 결과 분포가 달라진다**.
  - 해결: 최근 며칠의 관측을 확장 상태로 묶거나 (전략 1), 사용자 모델의 은닉 상태로 요약(전략 2).

확인 문제: MDP인가, 마르코프인가? (2/2)

- **(c) 슬롯머신(Week 2의 밴딧)**
  - 답: **MDP이지만 특수한 경우** — $\mathcal{S} = \{s_0\}$ (상태 1개), $P(s_0|s_0, a) = 1$ for all a.
  - 밴딧은 "**전이 없는 MDP**"로, 이 절의 모든 정의가 **퇴화된 형태로** 적용(FAQ 참고).
- **(d) 체스에서 다음 수를 고르는 문제(중반)**
  - 답: **MDP** — $\mathcal{S}$ = 합법적인 모든 판 배치(크기 $\sim 10^{40}$ 이상), $\mathcal{A}$ = 그 배치의 합법 수, P = 행동 후 판 배치(**결정적**이라 확률 1), R = 매 스텝 0, 체크메이트 시 ± 1.
  - 체스는 마르코프 성질이 **정확히** 성립하는 **희귀한 예** — "어떤 수순을 거쳐왔는지"가 다음 선택지에 영향을 주지 않으므로.

네 문항의 공통 기준

"현재 상태(관측)만 알면, 앞으로 일어날 것의 전체 확률분포가 결정되는가?" 아니라면 hidden state가 있다는 뜻이고, 상태 재설계(전략 1 또는 2)가 필요하다.



체스 대국 중인 판 — 마르코프 성질이 **정확히** 성립하는 희귀한 예: 다음 수는 오직 지금의 판 배치에만 의존하고, 거쳐온 수순과는 무관하다.

자주 하는 실수 (1/2): “관측”과 “상태”를 혼동

obs(관측)와 s(상태)는 역할이 다르다

관측(observation)

- 에이전트가 **볼 수 있는** 것.
- 마르코프 성질을 **보장하지 않는다** — Partially Observed MDP에서는 $o_t \neq s_t$ 다.

상태(state)

- 에이전트가 **알아야 하고, 알고리즘이 실제로 사용하는** 완전한 정보.
- 마르코프 성질이 **성립해야** 한다.

자주 하는 실수 (2/2): P 를 "한 번의 샘플"로 혼동

- $P(s'|s,a)$ 는 확률 **분포**이지 한 번의 결과가 아니다.

요약

"관측 $\neq$ 상태"(역할의 차이), "$P \neq$ 한 번의 샘플"(분포 vs 단일 결과). 두 실수는 모두 **확률적 MDP의 표기**와 **부분관측**을 혼동하는 것에서 온다.

실습: Gymnasium에서 $\mathcal{S}, \mathcal{A}$ 확인

이론의 다섯 요소를 실제 환경의 API와 대응시킨다. `FrozenLake-v1`(얼음 위를 미끄러지지 않게 건너는 문제)과 `CartPole-v1` 을 만들고, 각각의 $\mathcal{S}$ 와 $\mathcal{A}$ 를 출력:

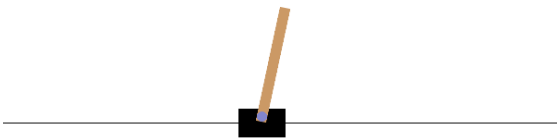
```
import gymnasium as gym

# --- FrozenLake: 이산상태행동 · 공간 ---
fl = gym.make("FrozenLake-v1", is_slippery=True)
obs, info = fl.reset(seed=0)
print("FrozenLake S:", fl.observation_space)   # Discrete(16)
print("FrozenLake A:", fl.action_space)         # Discrete(4)
print("초기 상태:", obs, "(4x4 격자칸 16 중하나 )")
fl.close()

print()

# --- CartPole: 연속상태 , 이산행동  ---
cp = gym.make("CartPole-v1")
obs, info = cp.reset(seed=0)
print("CartPole S:", cp.observation_space)    # Box(4,)
print("CartPole A:", cp.action_space)          # Discrete(2)
print("초기 상태:", obs, "[x, x_dot, theta, theta_dot]")
cp.close()
```

FrozenLake: **16개 이산 상태**(4×4 격자 칸), **4개 이산 행동**(상/하/좌/우) — $\mathcal{S}, \mathcal{A}$ 모두 유한이므로 P 를 16×4 크기의 표로 쓸 수 있다. CartPole: **연속 상태**($\mathbb{R}^4$ 의 제한된 범위), **2개 이산 행동** — P 를 표로 쓸 수 없고, `step()` 함수의 내부 물리 시뮬레이션(이산 시간 적분)으로만 정의된다. 이 차이가 Week 4–6(표 기반)과 Week 9–11(신경망 기반)의 분리를 **환경 수준에서** 보여준다.



CartPole-v1 렌더링 화면
연속 상태 $\mathbb{R}^4$, 이산 행동 2개



FrozenLake-v1 (4×4) 렌더링 화면
이산 상태 16개, 이산 행동 4개

실습: 마르코프 성질을 샘플링으로 직접 검증

마르코프 성질이 "성립한다"는 말이 **검증 가능한 수학적 주장**인지 직접 확인. 간단한 **결정적** 환경(2-경로 환경: 상태 0에서 행동 0이면 1로, 행동 1이면 2로 전이)을 만들고, **같은 상태 1**에 **다른 경로**(0→1 vs 0→2→1)로 도달한 뒤, 다음 상태 분포가 같은지 샘플링으로 확인:

```python
import numpy as np

class TwoPathsEnv:
    """0 -> {1, 2} -> 1 -> 1(self-loop) 결정적 MDP."""
    def __init__(self, seed=0):
        self.rng = np.random.default_rng(seed)
        self.s = 0
    def reset(self, seed=None):
        if seed is not None:
            self.rng = np.random.default_rng(seed)
        self.s = 0
        return self.s
    def step(self, a):
        if self.s == 0:
            self.s = 1 if a == 0 else 2
        elif self.s == 1:
            self.s = 1  # self-loop, 경로무관
        else:
            self.s = 1 if a == 0 else 2
        return self.s, 0.0, False, False, {}

# 마르코프검증 : 상태에 1 두경로로도달  , 다음전이분포비교
rng = np.random.default_rng(42)
dist_via_01, dist_via_021 = [], []
N = 10000
for _ in range(N):
    env = TwoPathsEnv(seed=rng.integers(0, 2**31)); env.reset()
    env.step(0); env.step(0)           # 0->1, 1->1
    dist_via_01.append(env.s)
for _ in range(N):
    env = TwoPathsEnv(seed=rng.integers(0, 2**31)); env.reset()
    env.step(1); env.step(0); env.step(0)  # 0->2->1, 1->1
    dist_via_021.append(env.s)

p1, p2 = np.mean(dist_via_01 == 1), np.mean(dist_via_021 == 1)
print(f"경로 0->1    : P다음(=1 | s=1) ≈ {p1:.4f}")
print(f"경로 0->2->1 : P다음(=1 | s=1) ≈ {p2:.4f}")
print(f"차이: {abs(p1-p2):.4f} 마르코프 ( 성질: 이어야함)")
assert abs(p1 - p2) < 0.01, "마르코프 성질위반 !"
print("✓ 두경로에서다음상태분포가일치—마르코프성질확인          ")
```

마르코프 검증: 핵심 패턴과 오차 분석

핵심 패턴: “같은 상태, 다른 경로, 다음 분포 비교”

이 패턴이 마르코프 성질 검증의 표준 절차다. 실습 노트북에서 이산/연속 환경 모두에서 더 자세히 한다.

- **결정적** 환경에서는 차가 **정확히 0**.
- **확률적** 환경에서는 샘플링 오차 ($\sim 1/\sqrt{N}$)가 있다.

마르코프 성질 검증 정의

실습: FrozenLake에서 P 를 실제로 추정하기

Gymnasium 환경의 P 는 내부 구현으로 감춰져 있지만, **샘플링으로 추정**할 수 있다:

$$P(s'|s,a) \approx \frac{N(s,a,s')}{N(s,a)}$$

((s,a) 에서 s' 로 전이한 횟수 / (s,a) 총 시도 횟수)로, 무작위 행동을 많이 하면 경험적 전이확률에 수렴.

```python
import gymnasium as gym
import numpy as np

env = gym.make("FrozenLake-v1", is_slippery=True)
env.action_space.seed(0)
n_states = env.observation_space.n   # 16
n_actions = env.action_space.n       # 4
# count[s][a][s'] = (s,a)->s' 전이횟수
count = np.zeros((n_states, n_actions, n_states), dtype=int)

for ep in range(2000):
    s, _ = env.reset(seed=ep)
    done = False
    while not done:
        a = env.action_space.sample()
        s2, r, term, trunc, _ = env.step(a)
        if s != 15 and s2 != 15:  # 터미널상태전이제외
            count[s, a, s2] += 1
        s = s2
        done = term or trunc

# 각 (s,a)에서 경험적분포행  ( 합이 1)
P_est = count / count.sum(axis=2, keepdims=True)
print("P_est shape:", P_est.shape, " (16 × 4 × 16)")
print("P_est[0, 0] 상태에서(0 left):",
      np.round(P_est[0, 0], 3))
env.close()
```

P 추정 결과: 모델 기반 vs 모델 없는

2000에피소드의 무작위 탐색으로 추정된 P 는, 실제 FrozenLake 구현(`is_slippery=True`일 때 intended 방향 2/3, 좌/우 1/6씩)에 **수렴**.

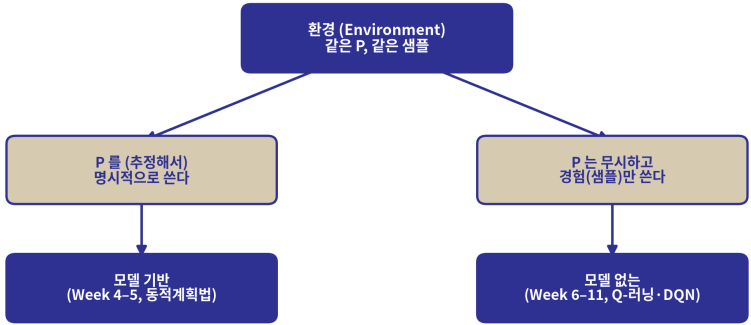
핵심: "P 의 사용 방식"이 알고리즘의 가족을 나눈다

이 P_{est} 를 Week 4의 동적계획법에 그대로 대입하면 **모델 기반** 알고리즘을 돌릴 수 있고, P 를 무시하고 **경험만** 쓰면 **모델 없는** 알고리즘(Week 6–)을 돌릴 수 있다.

같은 환경, 같은 샘플, 다른 "P 의 사용 방식"이 알고리즘의 가족을 나눈다.

모델 기반 vs 모델 없는

그림으로: 모델 기반 vs 모델 없는



같은 환경, 같은 샘플에서 추정된 P 를 **명시적으로 쓰느냐** (모델 기반, Week 4–5) **경험만 쓰느냐**(모델 없는, Week 6–11)가 이 학기 알고리즘 지도의 첫 갈림길이다.

이 절을 마무리하며: 강화학습의 “문법”

강화학습 문제 $=$ $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ $+$ **마르코프 성질**

1. 가치를 $V^{\pi}(s)$ 와 같은 “**상태에만 인자화된 함수**”로 정의할 수 있고(3.3),
2. **벨만방정식**이라는 한 스텝 재귀식으로 가치를 계산할 수 있고(3.3),
3. 정책평가 · Q-learning · DQN · PPO 까지 — **이 형식 위에서 쓰인 모든 알고리즘**을 도출할 수 있다.

마르코프 성질이 “깨진” 문제(POMDP)

이 “문법”의 **전제가 깨진** 것이므로, “상태를 재설계해서 성질을 다시 성립시키는 것”이 **항상 첫 번째 해결책**.

FAQ (1/2)

- 엄밀하게는 **거의 항상 근사**.
- **그렇다** — $\mathcal{S} = \{s_0\}$ (상태 하나)에서 어떤 행동을 해도 **항상 같은 상태로 "전이"**하는 특수한 MDP.

FAQ (2/2)

- **바뀌지 않는다** — 모든 $Q^\pi(s,a)$ 가 10배가 되지만, $\arg\max_a Q^\pi(s,a)$ 는 **변하지 않음**.
- **보상의 "스케일"은 안전, "형태"는 위험하다.**

FAQ (3/3)

- 터미널 상태: self-loop, 보상 0
- terminated: 터미널 상태 대응
- truncated: 학습 편의상 중단

3.2 누적 보상, 할인율, 그리고 실습

이 절의 흐름

수업의 흐름:

1. **리턴**(할인된 누적 보상)의 정의와 유한/무한 경우의 계산법.
2. “할인율 0.95”가 실제로는 “몇 스텝짜리 시야”를 뜻하는지 — **유효 시계(effective horizon)**로 바꾸는 법.
3. γ 를 바꾸면 “같은 환경에서 다른 행동을 **최적으로**” 만드는 예 — 할인율이 단순 정규화 장치가 아니라 **목표를 바꾸는** 매개변수임을 확인.
4. Gymnasium에서 CartPole로 할인된 리턴을 실제로 누적.

리턴(return)의 정의

강화학습 목표: 리턴 G_t 최대화

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

왜 할인율 γ 가 필요한가

1. **수학적으로**: $\gamma < 1$ 이면 보상이 유계(bounded)인 한 이 **무한급수가 항상 수렴**(기하급수). $\gamma = 1$ 이면 끝나지 않는 과업에서 리턴이 **무한대로 발산**할 수 있다.
2. **직관적으로**: "지금 당장의 보상 1"이 "10스텝 뒤의 보상 1"보다 더 가치 있다고 보는 경우가 많다(사람의 시간 선호, 금융의 이자율과 같은 방향).

닫힌 형태로 잘 풀리는 무한합

보상이 매 스텝 같은 값 R 으로 고정된 경우:

$$G_t = R + \gamma R + \gamma^2 R + \cdots = \frac{R}{1-\gamma}$$

- $\gamma = 0.9$, $R = 1$ 이면 $G_t = \frac{1}{0.1} = 10$ — “매 스텝 1을 무한히 받는 환경”의 리턴이 10이라는 뜻.

보상이 매 스텝마다 다른 일반적 경우: **한 에피소드가 유한한 스텝 T 뒤에 끝나면**(터미널 도달 뒤 보상은 모두 0) 무한합은 **유한합으로 잘린다**:

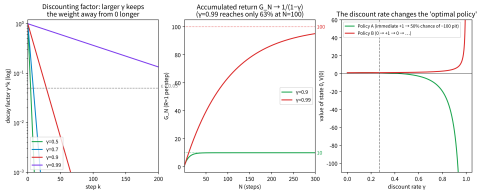
$$G_0 = \sum_{k=0}^{T} \gamma^k R_k = R_0 + \gamma R_1 + \gamma^2 R_2 + \cdots + \gamma^T R_T$$

매 스텝 보상 $R = 1$ 일 때: 유한합이 닫힌 형태로 접근

매 스텝 보상 $R = 1$ 일 때, 유한합 $G_N = \sum_{k=0}^{N-1} \gamma^k$ 이 닫힌 형태 $\frac{1}{1-\gamma}$ 로 어떻게 다가가는지:

N	$G_N\ (\gamma = 0.9)$	$G_N\ (\gamma = 0.99)$
5	4.10	4.90
10	6.51	9.56
20	8.78	18.21
50	9.95	39.50
100	10.00	63.40
∞	**10**	**100**

- $\gamma = 0.9$ 는 20스텝이면 이미 닫힌 형태의 **87.8%**, 100스텝이면 99.997%.
- $\gamma = 0.99$ 는 100스텝이 돼도 **63.4%**에 불과.



(가운데) 매 스텝 보상 1의 누적 리턴이 $1/(1-\gamma)$ 로 수렴.

결론

γ 가 **클수록 합산해야 하는 항의 수가 늘어** "더 장기적"이라는 말의 **구체적 수학적 의미**.

유효 시계: $\gamma = 0.95$ 는 몇 스텝짜리 시야인가

재정의

"할인 계수 γ^k 가 ϵ 아래로 떨어지는 첫 스텝 k 를 H 라 하자" — 이 H 를 유효 시계 (effective horizon)라고 부른다. "에이전트는 대략 H 스텝 까지 미래를 볼 수 있다"고 읽을 수 있다.

$\gamma^H = \epsilon$ 양변에 로그를 취하면:

$$H = \frac{\log \epsilon}{\log \gamma}$$

$\epsilon = 0.05$ 로 잡으면(= 5% 이하로 깎인 보상부터 "무시"):

γ	유효 시계 H ($\epsilon = 0.05$)	해석
0.5	4.3 → **약 4스텝**	5스텝 뒤 보상은 이미 3%
0.9	28.4 → **약 28스텝**	약 28스텝 뒤 보상은 5% 이하
0.95	58.4 → **약 58스텝**	CartPole 에피소드(500)의 ∼1/9
0.99	298.1 → **약 298스텝**	500스텝 에피소드의 60%까지 보임

유효 시계: 반대로 쓰기 + “5%”의 의미

반대로 쓰는 법(실전 팁)

유효 시계 공식을 뒤집으면 $\gamma = \epsilon^{1/H}$ 다. “에이전트가 H 스텝을 볼 수 있게 하라”고 정하면 γ 가 **자동으로** 결정된다:

- 500스텝짜리 에피소드를 다 보려면 $\gamma = 0.05^{1/500} \approx 0.994$.
- 100스텝짜리 환경이면 $\gamma = 0.05^{1/100} \approx 0.97$.

“**환경의 수명에 γ 를 맞춘다**”는 실전 관행이 바로 이 식에서 온다.

“5%로 깎였다”가 왜 “사실상 무시”인가?

H 스텝 **이후**에 들어오는 **모든** 보상의 합은, 그 직전 스텝의 보상에 $\frac{\epsilon}{1-\gamma}$ 를 곱한 것 **이상으로 클 수 없다**. $\gamma = 0.9$, $\epsilon = 0.05$ 이면 28스텝 이후의 **모든** 미래 보상의 합이 28스텝째 보상의 $0.05/0.1 = 0.5$ 배를 넘지 못한다.

“무한히 먼 미래”가 실제로는 **한 스텝짜리 상한**에 압축되는 것이다.

할인율은 “목표”를 바꾼다: 예 1

예 1 (당장 1 vs 5스텝 뒤 3): 매 스텝 보상 $R_0 = 1, R_1 = R_2 = R_3 = 0, R_4 = 3$ 인 환경.

γ	G_0 계산	G_0
1	$1 + 3$	4.00
0.9	$1 + 0.9^4 \times 3$	2.9683
0.5	$1 + 0.5^4 \times 3$	1.1875

같은 환경에서, γ 만 바꿔도 최선의 행동이 바뀌는 것이다.

할인율은 “목표”를 바꾼다: 예 2 (3상태 MDP)

예 2: 상태 0, 1, 2에, 각 상태에서 두 행동 A·B.

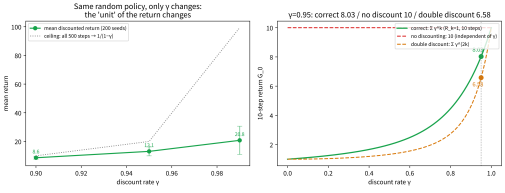
현재 상태	행동 A	행동 B
0	$\to$ 1, 보상 $+1$	$\to$ 1, 보상 $+1$
1	$\to$ 2(50%), 0(50%), $+2$	$\to$ 0(확정), 0
2 (루프)	자기 자신, -10	자기 자신, -10

- **정책 A: 상태 2로 넘어가 -10 루프**
- **정책 B: 상태 0으로 돌아가는 안전 루프**

예 2: γ 를 올리면 답이 뒤집힌다

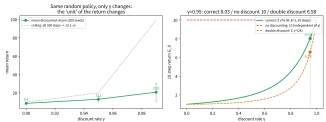
$$V^A = [-63.36,\ -71.51,\ -100], \qquad V^B(0) = \frac{1}{1-\gamma^2} = \frac{1}{0.19} \approx 5.26$$

- $\gamma = 0.9$ 에서는 **정책 A가 정책 B보다 훨씬 나쁘다**.
- γ **를 낮추면 답이 뒤집힌다**.
- **두 정책의 상태 0 가치 곡선이 교차**한다.



크로스오버 지점: “할인율은 목표를 바꾼다”의 의미

크로스오버 지점 $\gamma \gtrsim 0.27$에서 정책 B가 우세



핵심

같은 환경, 같은 두 정책인데 γ 만 바꾸면 “어느 정책이 옳은가”의 답이 뒤집힌다 ($\gamma \lesssim 0.27$ 에서 A, $\gamma \gtrsim 0.27$ 에서 B).

이것이 “할인율은 **목표를 바꾼다**”의 구체적 의미. γ 는 단순 **정규화 상수**가 아니라, **어떤 행동을 “옳은” 행동으로 만드느냐**를 정하는 **설계 변수**.

역사적 맥락

“현재 보상을 미래 보상의 기댓값에 더해 재귀적으로 정의한다”는 관점은 1950년대 **벨만(Bellman, 1957)**의 **동적계획법**에서 나왔다. “복잡한 문제를 한 번에 풀지 말고, 한 스텝만 보고 나머지는 부분 문제로 위임하라”는 통찰이 바로 3.3절의 벨만방정식인데, 그 식의 “한 스텝”을 **지금**과 **미래**로 나누어 가중하는 비율이 바로 γ 다. 할인율은 “수학의 편의”가 아니라, **결정 이론의 시간 가중(time discounting)**을 MDP에 이식한 것이다.

실습: CartPole에서 할인된 리턴 직접 누적

CartPole은 매 스텝 살아 있으면 $+1$ 을 주므로(1.3절), **보상이 전부 1**인 특수한 경우 — 리턴은 "얼마나 오래 버텼는가"의 **할인된 버전**.

```python
import gymnasium as gym

env = gym.make("CartPole-v1")
obs, info = env.reset(seed=0)

# 의MDP 다섯요소를실제로확인
print("상태 공간 S:", env.observation_space)   # 차원4 연속벡터
print("행동 공간 A:", env.action_space)          # 개2 이산좌우 (/)

total_return, gamma, discount = 0.0, 0.95, 1.0
for _ in range(10):
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = env.step(action)
    total_return += discount * reward   # 를G_t 직접누적
    discount *= gamma
    if terminated or truncated:
        break
print("스텝10 동안의할인된리턴   G_0:", round(total_return, 3))
env.close()
```

이 코드에서 reward가 매 스텝 즉시 보상 $R(s,a)$ 이고, 환경이 내부적으로 다음 상태로 전이시키는 규칙이 곧 $P(s'|s,a)$ 다 — Gymnasium은 이 전이확률을 코드로 감춰두고 step() 하나로 캡슐화한 것뿐, MDP의 정의와 정확히 대응.

CartPole 10스텝: $\gamma = 0.95$ 의 할인 효과

$$G_0 = 1 + 0.95 + 0.95^2 + \cdots + 0.95^9 = \frac{1 - 0.95^{10}}{1 - 0.95} \approx 8.02$$

실제로 출력되는 **8.025**가 이 값.

- 10스텝째 보상의 계수 $0.95^9 \approx 0.63$ 이므로, “10스텝”을 다 더한 10보다 **꽤 작다**.
- 이미 10스텝 만에 “할인의 효과”가 눈에 보인다.

베이스라인 잡기

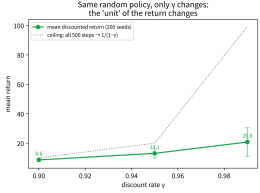
무작위 정책으로 200개 시드를 돌려, γ **의 크기가 “같은 환경”의 베이스라인 리턴을 어떻게 바꾸는지**를 확인.

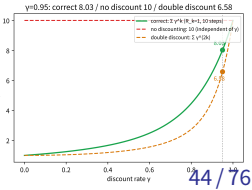
베이스라인: γ 가 리턴을 어떻게 바꾸나

```python
import gymnasium as gym
import numpy as np

env = gym.make("CartPole-v1")
for gamma in [0.9, 0.95, 0.99]:
    returns = []
    for seed in range(200):
        s, _ = env.reset(seed=seed)
        env.action_space.seed(seed)
        total, discount = 0.0, 1.0
        while True:
            a = env.action_space.sample()
            s, r, term, trunc, _ = env.step(a)
            total += discount * r
            discount *= gamma
            if term or trunc:
                break
        returns.append(total)
    returns = np.array(returns)
    print(f"gamma={gamma}: 무작위평균리턴   = {returns.mean():.2f}  "
          f"스텝{500 최대: {(1-gamma**500)/(1-gamma):.2f})")
env.close()
```

```
gamma=0.9: 평균리턴   = 8.62 최대 (: 10.00)
gamma=0.95: 평균리턴  = 13.07 최대(: 20.00)
gamma=0.99: 평균리턴  = 20.77 최대(: 99.34)
```





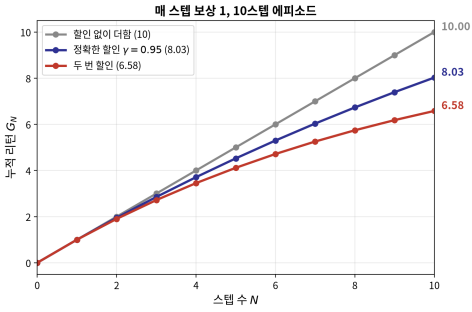
베이스라인 해석: 같은 γ 안에서만 비교

- 이는 “할인율이 보상의 **총량**이 아니라, **언제의 보상을 얼마나 헤아리는가**를 정한다”는 뜻.

주의: 서로 다른 “화폐 단위”

$\gamma = 0.9$ 의 “평균 8.62”와 $\gamma = 0.99$ 의 “평균 20.77”을 **같은 스케일로 비교하면 안 된다** — 서로 다른 **화폐 단위(할인 기준)**의 숫자.

같은 γ 안에서만 리턴의 크기를 비교하는 것이 정석.



할인율에 따른 리턴 비교 — 할인 없이 더함(10), 정확한 할인(8.03), 두 번 할인(6.58).

자주 하는 실수: 할인율을 빼먹기 / 두 번 할인

실수 1: 할인율을 빼먹고 그냥 보상을 다 더하기

실수 2: 할인을 두 번 적용하기(실수 1의 정반대)

확인 문제 (3.2)

1. $\gamma = 0.9, \epsilon = 0.05$ 일 때 유효 시계 H 를 $H < \log(0.05)/\log(0.9)$ 로 직접 계산. 로그를 쓰지 않고 $0.9^{28}, 0.9^{29}$ 를 직접 곱해서도 검증. 이 $H = 28$ 이 "에이전트는 28스텝 이후의 보상을 5% 이하로 취급한다"는 뜻과 일치하는지 확인.
2. 매 스텝 보상 $R = 1$ 인 환경에서, $\gamma = 0.9$ 와 $\gamma = 0.99$ 일 때 각각 "한 에피소드에서 받을 수 있는 **최대** 리턴"$(1/(1-\gamma))$을 계산. 이 차이가 "$\gamma = 0.99$ 를 쓰면 $\gamma = 0.9$ 때보다 **상대적으로** 더 먼 미래의 보상이 중요하다"는 뜻인지 한 문장으로 설명.
3. (개념) "할인율 0.95는 "5% 이자율""이라는 비유. 금융에서 "1년 뒤 1원"의 현재가치 $1/(1+r)$ 과 비교해, 강화학습의 γ 가 "이자율"과 **같은 방향**인지, **다르다**는 점(예: $\gamma = 1$ 이 "이자율 0"이 아니라 "미래 보상을 무한히 중시"하는 극단)을 각 한 문장으로.

FAQ (1/2): γ 설정

- **Q. γ 는 보통 얼마로 설정하나요?**
  - 0.9∼0.99 사이가 흔.
- **Q. 유한 에피소드 문제에서도 할인율이 꼭 필요한가요?**
  - 유한한 에피소드라면 $\gamma = 1$ 이어도 리턴이 **발산하지는 않는다**(합이 유한 개 항).

FAQ (2/2): γ 의 극단

- “당장의 스텝 **하나**의 보상뿐”을 본다는 뜻. $G_t = R_t$ 로 리턴이 즉시 보상과 같아지고, **근시안의 극단**.
- MDP는 퇴화하여 **다중팔 밴딧**(각 상태가 별개의 밴딧)이 된다.
- (1) **수학**: $\gamma = 1$ 이면 끝나지 않는 과업에서 발산할 수 있다. $\gamma < 1$ 은 **수렴을 수학적으로 보장**.
- (2) **실용**: 1에 가까울수록 리턴의 **분산**이 커지고, 수렴이 **느려져** 학습이 불안정해질 수 있다.
- “1에 가까울수록 좋은가”는 **환경의 수명**과 **알고리즘의 분산**을 함께 저울질해야 정해지는 **설계 판단**.

FAQ (3/3): “할인 두 번”과 오프바이원

- $G_0 = R_0 + \gamma R_1 + \gamma^2 R_2 + \cdots$ 의 정의에서, R_k 의 계수는 γ^k.
- **오프바이원 버그**

3.2절 한 줄 요약

리턴 G_t 는 “한 스텝의 보상이 아니라, 앞으로 받을 보상의 **할인된 합**”이고, 할인율 γ 는 그 합을 **수렴시키는 수학 장치**이면서 동시에, **어떤 보상을 중시하는지**를 정하는 **목표 변수**다.

같은 환경에서도 γ 를 바꾸면 “최적인 행동”이 바뀔 수 있다(예 2)는 점이, 할인율을 “단순 정규화 상수”가 아니라 **설계 결정**으로 봐야 하는 이유다.

3.3 가치함수와 벨만 기대방정식

이 절에서 어디로 가는지

흐름은 정확히 이 네 가지:

1. **가치함수** V^π, Q^π — "이 상태(상태+행동)에서 정책 π 를 따랐을 때 기대 리턴"을 정의.
2. **벨만 기대방정식** — 그 기대 리턴을 "지금 보상 + 다음 상태 가치(할인)"라는 **한 스텝짜리 재귀식**으로 변환.
3. **수치 계산** — 그 재귀식을 "값이 안 바뀔 때까지 반복 대입"하는 절차(반복적 정책평가)로 풀고, 손계산값과 검산.

가치함수: 리턴의 기댓값

정책(policy) 정의

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s] \qquad \text{(상태가치함수, state-value)}$$

"상태 s 에서 시작해서 π 를 계속 따른다면, 앞으로 받을 할인된 보상의 합이 **평균적으로** 얼마인가".

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a] \qquad \text{(행동가치함수, Q-함수)}$$

행동 가치함수 정의

표로 비교: V^π vs Q^π

	$V^\pi(s)$ 상태가치	$Q^\pi(s,a)$ 행동가치
고정하는 것	상태 s (행동은 π 가 정함)	상태 s **와 첫 행동** a
무작위성	행동 **과** 전이 모두	첫 행동만 제거, 전이만 남음
매기는 대상	$\mathcal{S}$ 에 하나씩	$\mathcal{S} \times \mathcal{A}$ 에 하나씩
“무엇을” 측정	그 상태에서 π 를 따를 때의 기대 리턴	a 를 한 다음 π 를 따를 때의 기대 리턴
주로 쓰이는 곳	“이 상태가 좋은가” (정책평가, 가치 추정)	“이 상태에서 어떤 행동이 좋은가” (정책비교/향상, Q-학습)

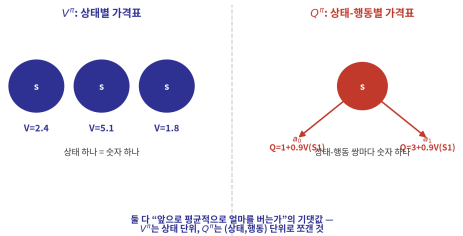
두 개를 왜 둘 다 정의하는가

둘 다 “리턴의 기댓값”인데, **어디를 고정한지가 다르다**. **동적계획법의 정책향상(policy improvement)**은 “지금 이 상태에서 **어떤 행동**이 가장 좋은가”를 물어보는데, 그 물음은 $V^\pi(s)$ 만으로는 답할 수 없고, 각 행동의 가치 $Q^\pi(s,a)$ 를 **행동마다 비교**해야 한다.

즉 Q 가 있으면 **비교**할 수 있고, V 만 있으면 **비교**할 수 없다. 그래서 둘 다 필요하다.

직관: “이 상태의 가격표”

- V^π: **상태별 가격표**
- Q^π: **상태-행동별 가격표**



중요: “실제 리턴”이 아니라 “기댓값”

이 숫자는 “**실제로 벌어지는 리턴**”이 아니라 그 **기댓값**이다. 같은 상태에서 시작해도 전이가 확률적이면 매 에피소드마다 리턴이 다르다 — $V^\pi(s)$ 는 “이 상태에서 출발하면 **평균적으로** 이 정도”를 말해주는 **요약 통계**다.

벨만 기대방정식: 무한합을 재귀식으로

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots = R_t + \gamma\big(R_{t+1} + \gamma R_{t+2} + \cdots\big) = R_t + \gamma G_{t+1}$$

즉 **리턴은 "지금 당장의 보상 + 할인된 다음 시점 리턴"으로 재귀적으로 쓸 수 있다**. 이 관찰을 V^π의 정의에 대입하면 (기댓값의 **선형성**을 이용):

$$V^\pi(s) = \mathbb{E}_\pi\big[R_t + \gamma G_{t+1} \mid s_t = s\big] = \mathbb{E}_\pi\big[R_t + \gamma V^\pi(s_{t+1}) \mid s_t = s\big]$$

이게 (Bellman expectation equation)

무한히 먼 미래까지 다 더해야 할 것 같던 $V^\pi(s)$ 가, "**지금 보상 + 다음 상태의 가치(할인됨)**"이라는 **한 스텝짜리 재귀식**으로 압축된다.

왜 G_{t+1} 을 $V^{\pi}(s_{t+1})$ 로 바꿀 수 있는가

- G_{t+1} 은 **구체적 경로** 하나(실제로 벌어진 특정 보상 열)의 리턴이고, $V^{\pi}(s_{t+1})$ 은 "상태 s_{t+1} 에서 출발하는 **모든 경로**의 리턴의 **평균**"이다.
- 이 둘이 같아도 되는 이유는 **마르코프 성질**(3.1절). 다음 상태 s_{t+1} 이 정해졌다면, 그 뒤에 이어질 보상 열의 분포는 **어떻게 s_{t+1} 에 왔는지에 의존하지 않는다**.
- 따라서 "여기까지 어떤 경로를 통해 왔느냐"는 정보 전체를 버리고 "지금 s_{t+1} 에 있다"는 사실 **하나**로만 조건부 지을 수 있다 — 그 조건부 기댓값이 바로 $V^{\pi}(s_{t+1})$ 의 정의.

요약

벨만방정식은 결국 **마르코프 성질을 "한 스텝만" 적용한 결과**인 셈이다. 3.1절의 성질이 왜 강화학습의 뼈대인가를, **수식 한 줄**이 정확히 보여준다.

이 재귀식이 얼마나 강력한가 — 상태가 100만 개여도

- 벨만방정식은 “상태 s 의 가치를 알려면 **인접 상태들** s' 의 가치만 알면 된다”고 말한다.
- 즉 100만 개의 상태를 모두 한 번씩 “인접 상태의 가치에 지금 보상을 더하는” 일로 계산 — **경로 탐색이 상태별 한 스텝 연산으로** 압축.

동적계획법의 핵심 아이디어

이 “무한 경로 → 인접 상태만”이 바로 **동적계획법 (dynamic programming)**이라는 이름의 **핵심 아이디어**이며, 다음 장의 **정책평가 알고리즘 전체**가 이 한 줄의 직접적 결과물이다.

무한 경로를 상태 수로 줄이는 재귀식

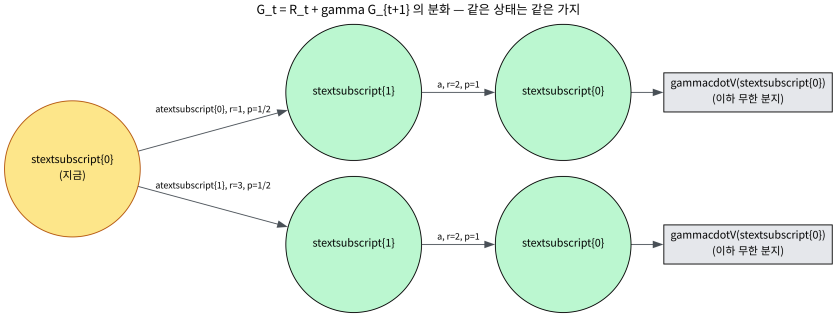
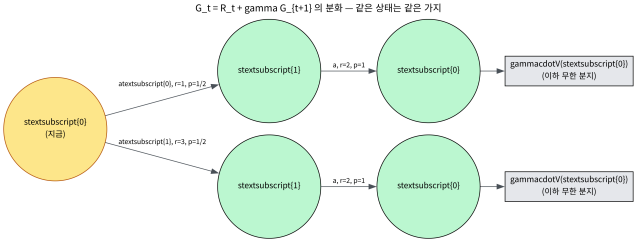


그림 읽어보기: “같은 상태는 같은 가지”

- **마르코프 성질: 같은 상태는 같은 기댓값**

핵심 작업

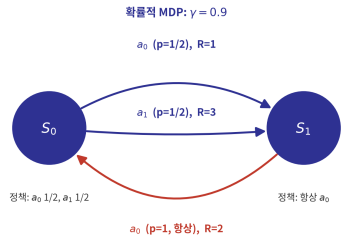
“**같은 상태를 하나의 노드로 병합**”하는 작업이 벨만방정식의 기하학적 모습이다. **경로 수(무한)를 상태 수 (여기서는 2개)로 줄이는** 것이다.



확률적 MDP로 손계산: MDP 정의

MDP 정의: 상태 2개(S_0, S_1), $\gamma = 0.9$.

- S_0 에서 정책 π 는 a_0 를 1/2, a_1 를 1/2 확률로 고른다.
  - a_0: 즉시 보상 $R = 1$, **반드시** S_1 로 전이.
  - a_1: 즉시 보상 $R = 3$, **반드시** S_1 로 전이.
- S_1 에서 정책 π 는 **항상** a_0를 고른다:
  - a_0: 즉시 보상 $R = 2$, **반드시** S_0 로 전이.



손계산: Q 부터, V 는 가중평균

Q^π **부터 계산** — 각 (상태,행동)의 행동가치는 "그 행동의 즉시 보상 + 다음 상태의 가치(할인)":

$$Q^\pi(S_0, a_0) = 1 + 0.9\, V^\pi(S_1), \qquad Q^\pi(S_0, a_1) = 3 + 0.9\, V^\pi(S_1)$$

$$Q^\pi(S_1, a_0) = 2 + 0.9\, V^\pi(S_0)$$

V^π **는 정책의 행동을 확률로 가중평균**:

$$V^\pi(S_0) = \tfrac{1}{2} Q^\pi(S_0, a_0) + \tfrac{1}{2} Q^\pi(S_0, a_1)$$

$$= \tfrac{1}{2}(1 + 0.9 V^\pi(S_1)) + \tfrac{1}{2}(3 + 0.9 V^\pi(S_1)) = \tfrac{1}{2} \cdot 4 + 0.9\, V^\pi(S_1) = 2 + 0.9\, V^\pi(S_1)$$

$$V^\pi(S_1) = Q^\pi(S_1, a_0) = 2 + 0.9\, V^\pi(S_0)$$

손계산: 연립방정식 풀기와 검산

$$V^\pi(S_0) = 2 + 0.9\big(2 + 0.9\,V^\pi(S_0)\big) = 2 + 1.8 + 0.81\,V^\pi(S_0) = 3.8 + 0.81\,V^\pi(S_0)$$

$V^\pi(S_0)(1 - 0.81) = 3.8$ 이므로 $V^\pi(S_0) = 3.8/0.19 = \mathbf{20}$. 다시 대입하면 $V^\pi(S_1) = 2 + 0.9 \cdot 20 = \mathbf{20}$.

- $Q^\pi(S_0, a_0) = 1 + 0.9 \cdot 20 = 19$, $Q^\pi(S_0, a_1) = 3 + 0.9 \cdot 20 = 21$.
  $\frac{1}{2}(19) + \frac{1}{2}(21) = 20 = V^\pi(S_0)$ ✓
- $Q^\pi(S_1, a_0) = 2 + 0.9 \cdot 20 = 20 = V^\pi(S_1)$ ✓

이 예가 보여준 것: 기대값으로 값이 결정된다

- V**는 정책 가중** Q**의 합과 일치**
- **기대 보상 2로 S0와 S1 가치 동일**

상태의 가치는 “기대(평균) 보상”으로 결정된다

상태의 가치는 “매 스텝 **보장되는** 보상”이 아니라 **정책이 정하는 기대(평균) 보상**에 의해 결정된다.

손으로 한 번: 2상태 순환 MDP (결정적)

2상태 순환 MDP: S_0**(R=1),** S_1**(R=2),** $\gamma = 0.9$

리턴의 무한 등비급수를 **직접 닫힌 형태로** 풀면:

$$V(S_0) = R_0 + \gamma R_1 + \gamma^2 R_0 + \cdots = \frac{R_0 + \gamma R_1}{1 - \gamma^2} = \frac{1 + 0.9 \times 2}{1 - 0.81} \approx \mathbf{14.737}$$

$$V(S_1) = \frac{R_1 + \gamma R_0}{1 - \gamma^2} = \frac{2 + 0.9 \times 1}{1 - 0.81} \approx \mathbf{15.263}$$

벨만 기대방정식 $V(s) = R(s) + \gamma V(s')$ 을 실제로 만족하는지 **검산**:

$$R_0 + \gamma V(S_1) = 1 + 0.9 \times 15.263 \approx 14.737 = V(S_0) \checkmark$$

$$R_1 + \gamma V(S_0) = 2 + 0.9 \times 14.737 \approx 15.263 = V(S_1) \checkmark$$

두 방식, 같은 답: 닫힌형 vs 반복 대입

- **“무한급수를 닫힌 형태로 직접 계산”과 “재귀식을 값이 안 바뀔 때까지 반복 적용”이라는 서로 다른 두 방법이 같은 답에 도달**한다.

이 “같은 답”은 우연이 아니다 — $\gamma < 1$ 이면 벨만방정식은 **유일한 고정점**을 갖고, 어떤 초기값에서 출발해도 그 곳으로 수렴(다음 프레임의 수축 연산 논증).

반복 대입: 재귀식을 푸는 절차 (다음 장 예고)

값이 안 바뀔 때까지 반복 대입

```
def policy_evaluation_value_iteration(P, R, policy, gamma,
                                      max_iter=200, theta=1e-6):
    # P[s][a] = [(prob, next_state), ...]
    # R[s][a] = 즉시보상  ; policy[s] = 상태에서 s 고르는행동
    n = len(P)
    V = [0.0] * n                       # 으로0 시작초기값은 ( 아무거나)
    for it in range(1, max_iter+1):
        V_new = list(V)                 # 동기식: 이전"" 를V 쓴다
        delta = 0.0
        for s in range(n):
            a = policy[s]
            v_new = R[s][a] + gamma * sum(p * V[s2] for p, s2 in P[s][a])
            delta = max(delta, abs(v_new - V[s]))
            V_new[s] = v_new
        V = V_new
        if delta < theta:               # 값이거의 () 안바뀌면멈춤
            break
    return V, it
```

동기식(synchronous)으로 쓴 이유

동기식(synchronous)으로 쓴 이유

매 반복에서 **모든** 상태가 **이전** 반복의 V 를 참조해 갱신된다. 그러면 "반복을 k 번 돌렸을 때 반영된 것"이 **정확히 "미래 k 스텝까지의 리턴"**이 된다 (반복 1 = 즉시 보상만, 반복 2 = 2스텝까지, . . .). 갱신을 **그 자리 바로** 하는 방식(Gauss-Seidel)도 같은 답으로 수렴하지만, "반복 횟수 $=$ 보는 미래 스텝 수"라는 **직관이 조금 흐려진다**. 이번 절에서는 그 직관이 핵심이라 **동기식**으로 쓴다.

- $\gamma < 1$ 이면 이 반복은 **보증이 있다**: 모든 $V(s)$ 가 벨만방정식의 **고정점(fixed point, 좌변 $=$ 우변인 값)**으로 수렴 — 아래 "고정점" 섹션에서 증명.

2상태 예: 반복 대입의 수렴

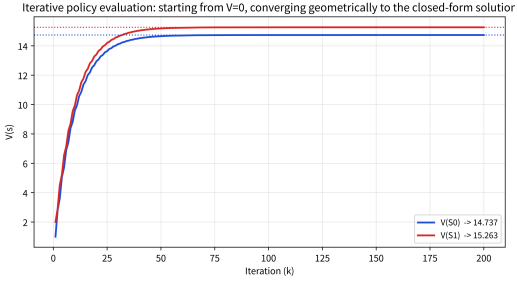
위 2상태 예($R_0 = 1, R_1 = 2, \gamma = 0.9$)를 이 함수로 $V_0 = V_1 = 0$ 에서 200번 돌리면:

반복	V_0	V_1
1	1.000	2.000 (“즉시 보상만” 반영)
10	9.598	9.941 (10스텝까지 반영)
50	14.661	15.185
200	**14.737**	**15.263** (**닫힌 형태와 정확히 일치**)

“반복 k 번 $\approx$ 미래 k 스텝까지의 리턴”

이 직관이 가치 반복의 **본질**. 반복을 한 번 더 하면 “한 스텝 더 먼 미래”가 반영.

γ^k 가 빠르게 0으로 줄므로(여기서 $\gamma = 0.9$, 10스텝 $0.9^{10} \approx 0.35$, 50스텝 ≈ 0.005), **수렴은 기하급수적으로 빨라진다**. 수렴 “속도”를 좌우하는 것이 바로 γ — 1에 가까울수록 (장기적일수록) **수렴이 느려지는** 이유이기도 하다.



반복적 정책평가 수렴 곡선 — $V = 0$ 에서 시작해 기하급수적으로 닫힌해로 접근.

왜 $\gamma < 1$ 이면 수렴하는가 (증명 개요)

벨만 연산자를

$$\mathcal{B}^{\pi} V(s) = R(s, \pi(s)) + \gamma \, \mathbb{E}_{s'}[V(s')]$$

라고 쓰자. 두 값 함수 V, W 에 대해 $\gamma < 1$ 이면:

$$\left\| \mathcal{B}^{\pi} V - \mathcal{B}^{\pi} W \right\|_{\infty} \leq \gamma \, \| V - W \|_{\infty}$$

($\| \cdot \|_{\infty}$ = 최대 절댓값, "어떤 상태에서 가장 많이 다르냐")

수축 연산 + 바나흐 고정점 정리

즉 벨만 연산자는 γ 를 계수로 줄이는 수축 (squeeze) 연산. 바나흐 고정점 정리(Banach fixed-point theorem)에 의해, 수축 연산은 고정점이 딱 하나 존재하고, 어떤 초기값에서 출발해도 그 고정점으로 수렴한다.

이 한 줄의 정리가 "가치 반복이 항상 (고유한) 답으로 수렴한다"는 **보장의 전부**.

- $\gamma = 1$ **이면** 수축 상수가 1이 되어 이 보장이 **사라진다** — 수렴하지 않거나, 유한 에피소드여도 경계가 무한대가 되면 발산할 수 있다.
- 3.2절에서 "$\gamma < 1$ 이면 수렴"이라 말한 것이, 여기서 **수학적으로 왜 그런가**가 보인다.

고정점의 유일성

$\gamma < 1$ 이면 벨만방정식을 만족하는 V^{π} 는 **정확히 하나뿐** — "이 정책의 가치함수"라는 개념이 **의미가 있는(well-defined)** 것의 근거.

“벨만방정식”이라는 이름의 유래 (역사)

리처드 벨만 (Richard Bellman, 1920–1984)

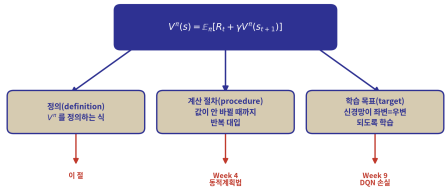
“Bellman equation”은 강화학습만의 개념이 아니다

최적제어, 금융공학, 확률과정 통계 등 “시간에 따라 상태가 변하며 **순차적으로 결정**해야 하는 문제” 전반에서 쓰이는, 20세기 수학의 **가장 널리 인용된 재귀식 중 하나**.

강화학습에서 이 방정식이 “**기대**”(expectation, 이 절) 버전과 “**최적**”(optimality, max 이 붙은, Week 4) 버전으로 나뉘어 등장하는 것뿐, **뼈대는 벨만이 70여 년 전에 세운 것과 정확히 같다**.

벨만방정식의 세 가지 읽기방식

1. **정의(definition)로서**: “이 정책 π 의 상태가치는 다음 조건을 만족하는 **(유일한) 함수**다.” — 이 절의 관점. V^π 를 **정의하는** 방정식.
2. **계산 절차(procedure)로서**: “값이 안 바뀔 때까지 $V \leftarrow \mathcal{B}^\pi V$ 로 반복 대입.” — 위 정책평가 코드. **다음 장 동적계획법**의 알고리즘.
3. **DQN 손실은 벨만방정식 좌우변 오차**



같은 재귀식을 어떤 형태로 계산·학습하는가 — 이 학기 알고리즘 지도

자주 하는 실수 (3.3)

실수 1: “다음 상태의 가치”에 할인율 γ 를 빼먹기

가치함수 vs 실제 리턴

- $V^{\pi}(s)$는 기댓값

해는 하나

- 고정점 유일

FAQ (1/2): V 와 Q 의 연결, 그리고 "최적"

- $V^\pi(s) = \sum_a \pi(a|s)\, Q^\pi(s,a)$ — 정책이 각 행동을 고를 **확률로** Q^π **를 가중평균**.
- 반대로 $Q^\pi(s,a) = R(s,a) + \gamma\, \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^\pi(s')]$ — "이번 행동만 a 로 **강제로 고정**하고, 그 다음부터는 π 를 따른다".

FAQ (2/2): “기대”는 어디에? 왜 “풀어야” 하나?

- 방정식 안의 $\mathbb{E}_\pi$ 가 바로 “기대”.
- **기댓값은 정책과 전이의 무작위성에 대해 취한다**
- **정의는 무한 경로 평균, 계산은 불가능**
- **벨만방정식은 무한 경로를 한 스텝으로 압축**

FAQ (3/3): γ 의 극단과 정책평가

- $\gamma = 0$ 이면 $V^{\pi}(s) = R(s, \pi(s))$ — “지금 당장의 보상만”이 가치. 에이전트가 **완전히 근시안적**(myopic)이 되어 미래를 전혀 고려하지 않음.
- **반복 대입은 정책평가**

한 줄

이 절의 10줄 코드가 다음 장의 알고리즘 전체의 **뼈대**.

3.3절 마무리

MDP는 “지금 한 행동이 미래의 상태 자체를 바꾼다”는 사실 하나를 Week 2의 밴딧에 추가한 것뿐이지만, 이 한 가지 차이가 강화학습을 **훨씬 더 어렵고, 훨씬 더 흥미로운** 문제로 만든다.

- **가치함수** $V^\pi(s), Q^\pi(s,a)$ = 리턴 G_t 의 **기댓값** (상태 고정 / 상태+첫 행동 고정).
- **벨만 기대방정식** = 마르코프 성질을 “한 스텝만” 적용한 결과 — 무한합을 “지금 보상 + 할인된 다음 상태 가치”의 **재귀식**으로 압축.
- **반복 대입(정책평가)** = $\gamma < 1$ 이면 **고정점 하나로 수렴**(수축 연산 + 바나흐) — 다음 장의 알고리즘 전체의 뼈대.

3주차 정리

1. **MDP의 다섯 요소 + 마르코프 성질 = 강화학습의 "문법"** — $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ 가 "게임의 규칙 전체"를 닫고, "현재 상태만 알면 미래 분포가 결정되는가"가 성립의 잣대.
2. **리턴 G_t 와 할인율 γ** — γ 는 **수렴을 보장하는 수학 장치**이자 **어떤 보상을 중시하는지 정하는 설계 변수**. "할인율은 발산을 막는 장치"는 반쪽이고, "할인율은 **목표를 바꾼다**"가 나머지 반쪽.
3. **가치함수와 벨만 기대방정식** — V^π, Q^π 는 "동전의 양면"이고, 벨만방정식은 **마르코프 성질을 한 스텝만 적용**한 결과. 반복 대입(정책평가)은 다음 장 동적계획법의 **뼈대**.

다음 주 — Week 4. 동적계획법 (DP)

이번 장의 MDP를 실제로 "**풀기**" 시작한다. 전이확률 P 를 정확히 안다는 전제(모델 기반) 아래, 벨만방정식의 **고정점**을 계산하는 **정책평가**와, 최적 정책을 찾는 **정책반복·가치반복**을 다룬다. 이번 장은 "문제를 **정의하는** 장"이었고, Week 4부터는 그 정의를 "**풀어가는** 장"으로 넘어간다.